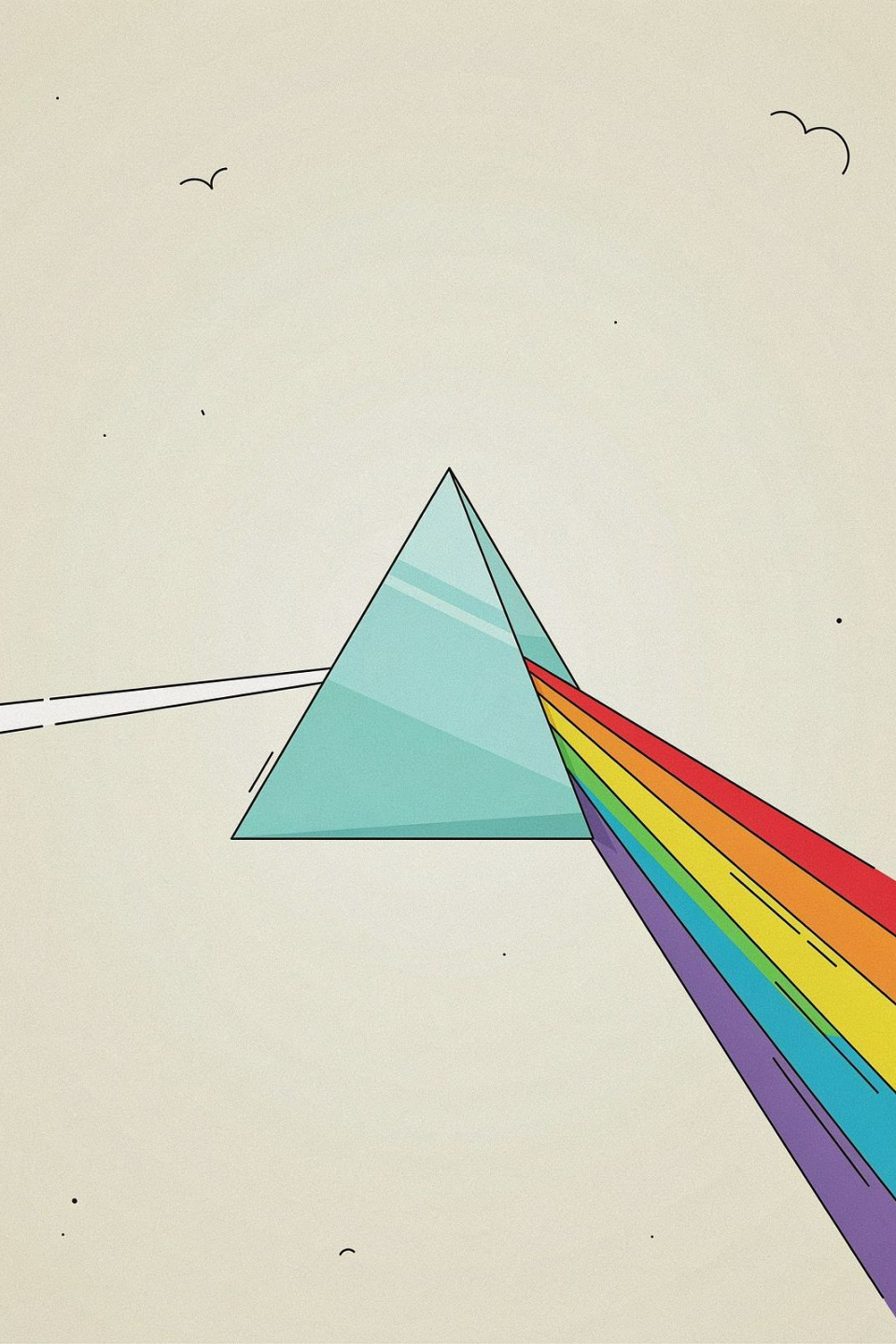




Memory, Context, Planning & Reasoning

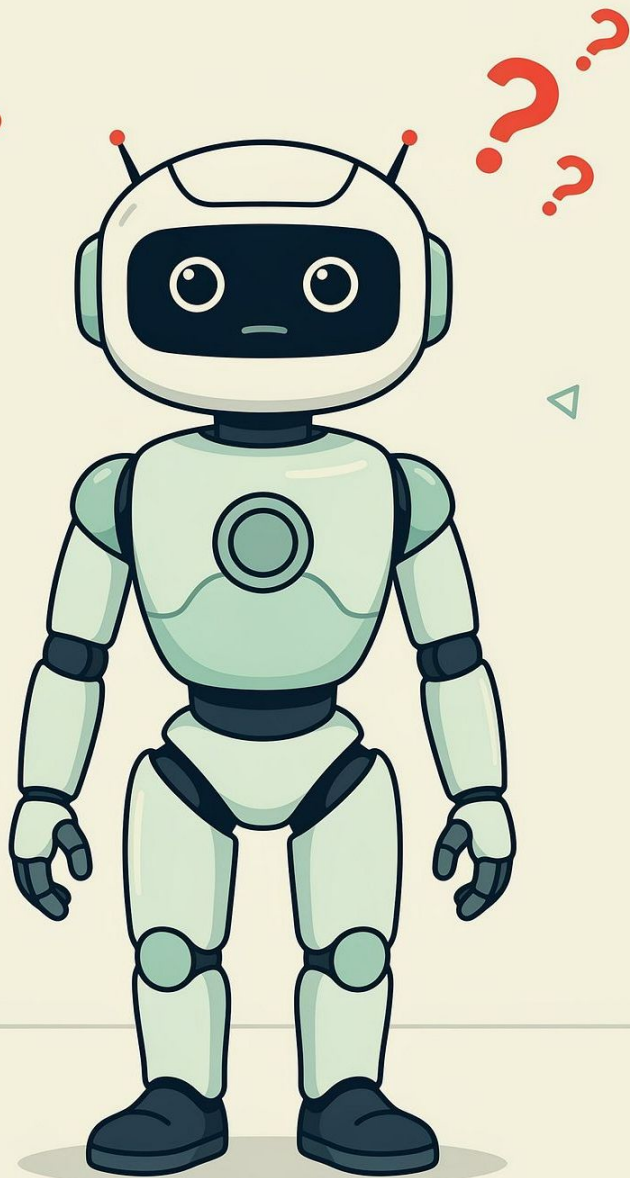
Making Agents Remember, Think, Plan and Adapt

Our Journey So Far

We've progressively built up the agent stack — from raw LLM calls to tool-using agents. Now we tackle the harder challenge: making agents capable of handling **complex, multi-step tasks** that require state, memory, and planning.



📌 **Key question:** How do we move from a tool-using agent to an agent capable of handling a genuinely complex task?



The Limitation of a Stateless Agent

A simple interaction works fine



But then the user continues...

User: "Continue what you were doing."

Agent: "What were we doing?"

An LLM processes each prompt independently. It has no automatic, built-in awareness of what happened before. **Persistent memory is not a property of the model — it is a property of your application.**

What Does "Memory" Actually Mean?

Memory is not a single concept. Treating it as one leads to poor architectural decisions. Each type serves a different purpose and requires a different engineering approach.

Type	Purpose	Example
Conversation memory	Previous dialogue turns	Chat history in the prompt
Working memory	Current task state	Structured state object mid-task
Short-term memory	Recent observations	Last 5 tool results
Long-term memory	Info retained across tasks	User preferences, past decisions
External memory	Persistent storage	Vector DB, files, SQL database

📌 **Key principle:** Memory is an engineering decision, not an automatic property of an LLM.

Context ≠ Memory

This is one of the most important distinctions in agent engineering. Conflating context and memory leads to brittle, expensive, and unreliable systems.

Context

What the model can **see right now**. Everything inside the context window during a single inference call. It disappears after the call completes.

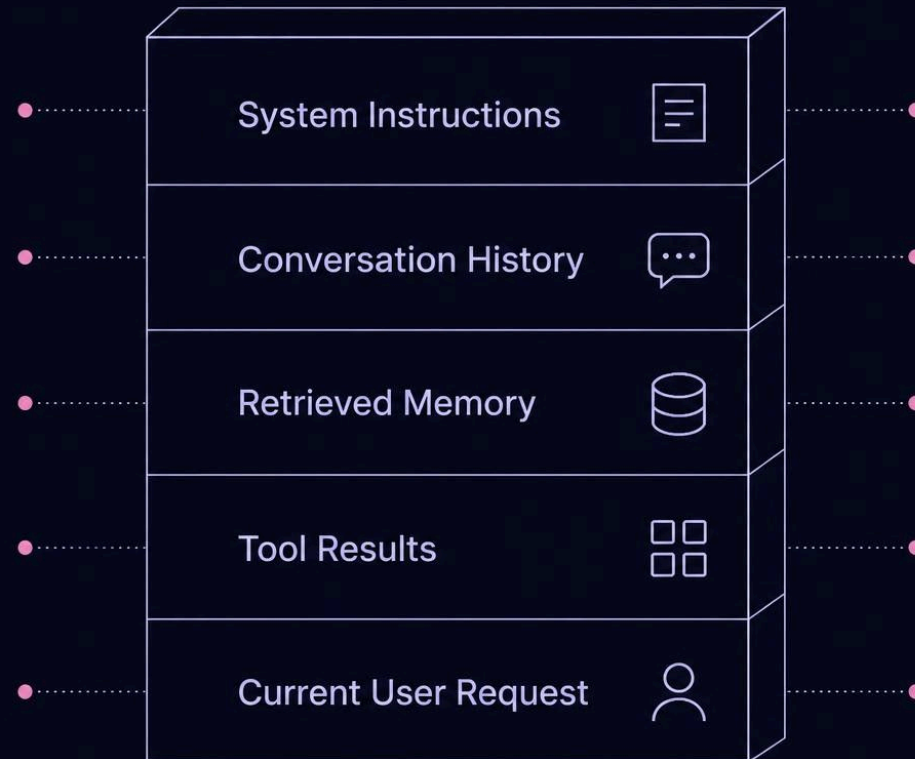
Memory

What the **application decides to retain** and potentially retrieve later. Memory is managed by your code — not the LLM. The LLM only ever sees what you choose to put into context.



The Context Window

Every token sent to the model consumes finite context space. Understanding what occupies that space is the first step toward building efficient agents.



⚠ Critical insight: More context does not automatically produce better reasoning. Irrelevant or excessive context can actively degrade model performance.



Context Engineering

The naive approach — giving the LLM everything available — is an anti-pattern. Instead, treat context construction as a deliberate engineering discipline. The goal is to provide *exactly* what the agent needs to reason correctly, and nothing more.

1 What does the agent need?

Identify the minimum information required for the current reasoning step.

2 What is relevant?

Filter aggressively. Irrelevant content adds noise, not signal.

3 What should be retrieved vs. omitted?

Only retrieve memory that directly bears on the current task.

4 What should be summarized?

Compress older context rather than including it verbatim.

Working Memory: The State Object

Rather than expecting the LLM to reconstruct task state from conversation history, introduce an explicit **state object** managed by your application. The LLM reasons over it; your code owns it.

```
state = {  
  "employee": {  
    "name": "Ramesh Kumar",  
    "role": "Senior Python Engineer"  
  },  
  "equipment": {  
    "laptop": "pending",  
    "badge": "issued"  
  },  
  "completed_steps": ["HR form", "badge"],  
  "pending_steps": ["laptop", "accounts"],  
  "approvals": ["manager_sign_off"],  
  "errors": []  
}
```

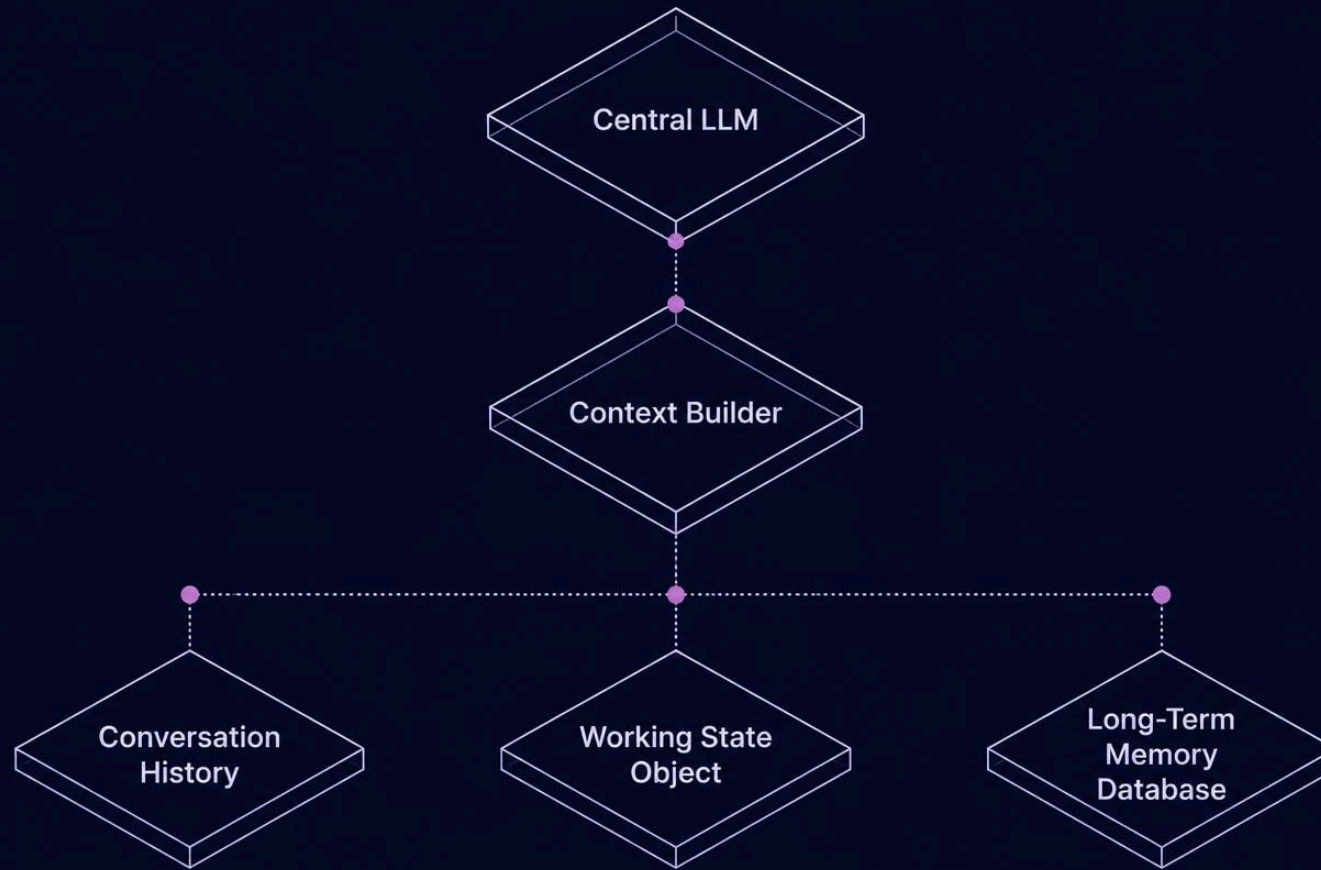
Why this matters

The LLM does not need to remember everything across turns. The **application remembers the state**. The LLM's job is to reason over the current snapshot — not to reconstruct history from scratch on every call.

This dramatically reduces context size, improves reliability, and makes agent behavior auditable.

Memory Architecture

A well-structured agent separates concerns across distinct memory stores. The context builder is responsible for assembling the right combination of inputs before each LLM call.



Each store has different latency, cost, and reliability characteristics. Choosing the right store for each type of information is a core architectural decision.

When Should We Use Long-Term Memory?

A vector database is not a default component. Adding one without a clear business requirement introduces complexity, latency, and cost. Ask the engineering question first.

Use persistent memory when the workflow requires:

→ **Cross-session information**

The agent must recall facts from a previous session.

→ **User preferences**

Personalization that must survive conversation resets.

→ **Previous decisions**

Avoiding repeated reasoning over the same question.

→ **Historical cases**

Learning from past similar workflows.

❓ **Engineering question:** What specific business requirement justifies the cost and complexity of persistence? If you cannot answer this, don't add it yet.

Memory Strategies

There is no single correct memory strategy. The right choice depends on task complexity, session length, cost constraints, and how much information genuinely needs to be preserved.

A — Full Conversation

Include the entire dialogue. Simple to implement, but context grows linearly and eventually becomes unmanageable.

B — Sliding Window

Keep only the most recent N messages. Loses older context, but bounds cost and remains simple.

C — Summarization

Periodically compress older turns into a summary. Balances retention and context size.

D — Structured State

Store facts in JSON or a database. Highly reliable; the LLM reads structured snapshots rather than raw history.

E — Retrieval

Fetch only the most relevant historical information at query time. Scales to large memory stores.

The Planning Problem

Single-action tasks are easy. Real enterprise workflows are not. The moment a goal requires multiple coordinated steps, an agent needs a plan — not just the next action.

Simple request

"Tell me the weather."

One action. One tool call. Done.

Complex request

"Onboard a new employee."

01

Collect & validate information

02

Check equipment availability

03

Request approvals

04

Create accounts & schedule induction

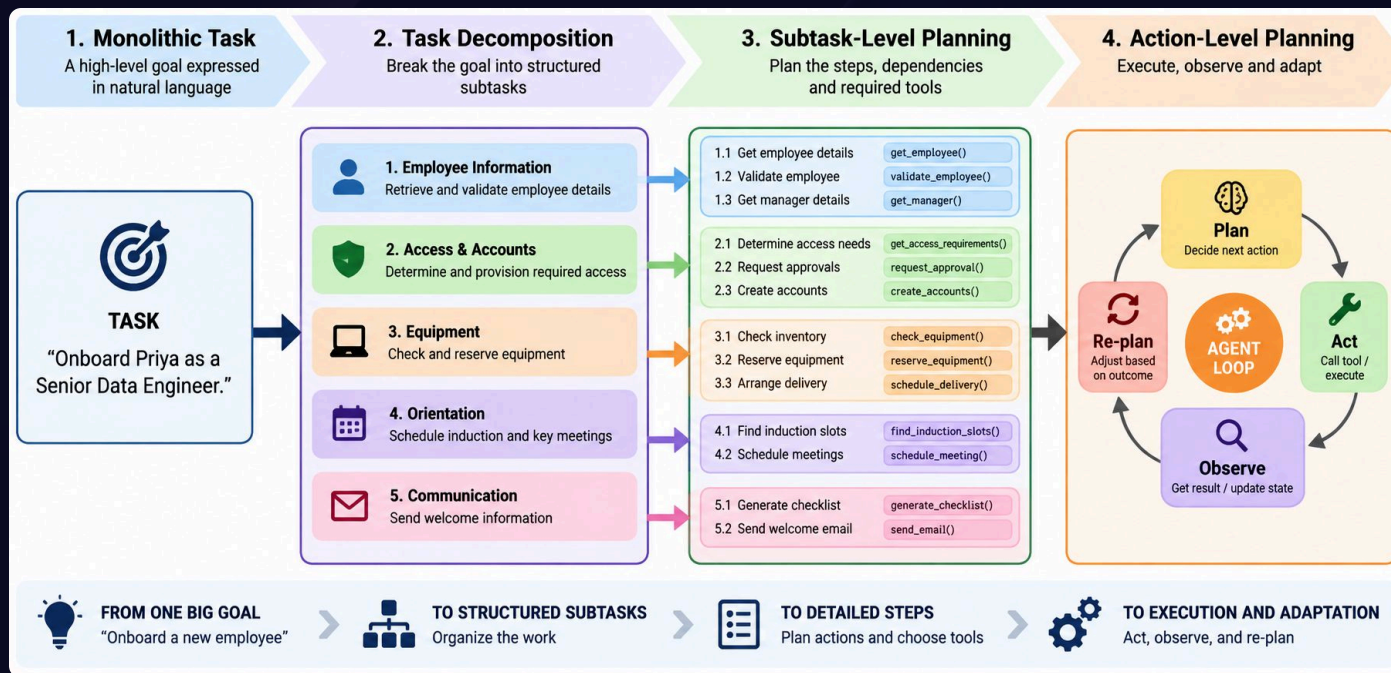
❏ **Complex goals require decomposition.** An agent cannot act on a complex goal directly — it must first break it into executable tasks.



Task Decomposition

The agent's first job when receiving a complex goal is to convert it into a structured hierarchy of executable tasks.

This transforms an ambiguous instruction into an actionable work plan.



Decomposition makes

1. complex goals tractable
2. enables parallel execution where possible
3. provides clear checkpoints for observation and replanning.

Planning vs. Execution

These are distinct cognitive roles that should be architecturally separated. Mixing them in a single agent makes behavior harder to debug, test, and control.

Planner

"What needs to happen?" Converts the goal into an ordered sequence of tasks. Reasons about dependencies and constraints.

Executor

"Do it." Carries out the next action in the plan. Calls tools, writes outputs, interacts with systems.

Observer

"Did it work?" Captures the result of the action — success, failure, or partial outcome — and feeds it back into the loop.



This separation becomes the foundation for the **Planner → Executor → Reviewer** architecture introduced in the next deck.

Planning Strategies

Not all planning strategies are equally suited to uncertain, real-world workflows. The choice of strategy directly affects how resilient the agent is to unexpected failures and changing conditions.

1. Upfront Planning

Generate the complete plan before executing any step. Simple and auditable, but brittle — the plan breaks when reality diverges from assumptions.

2. Replanning

Execute, observe the result, then revise the remaining plan. More adaptive. Handles failures gracefully. Preferred for workflows with external dependencies.

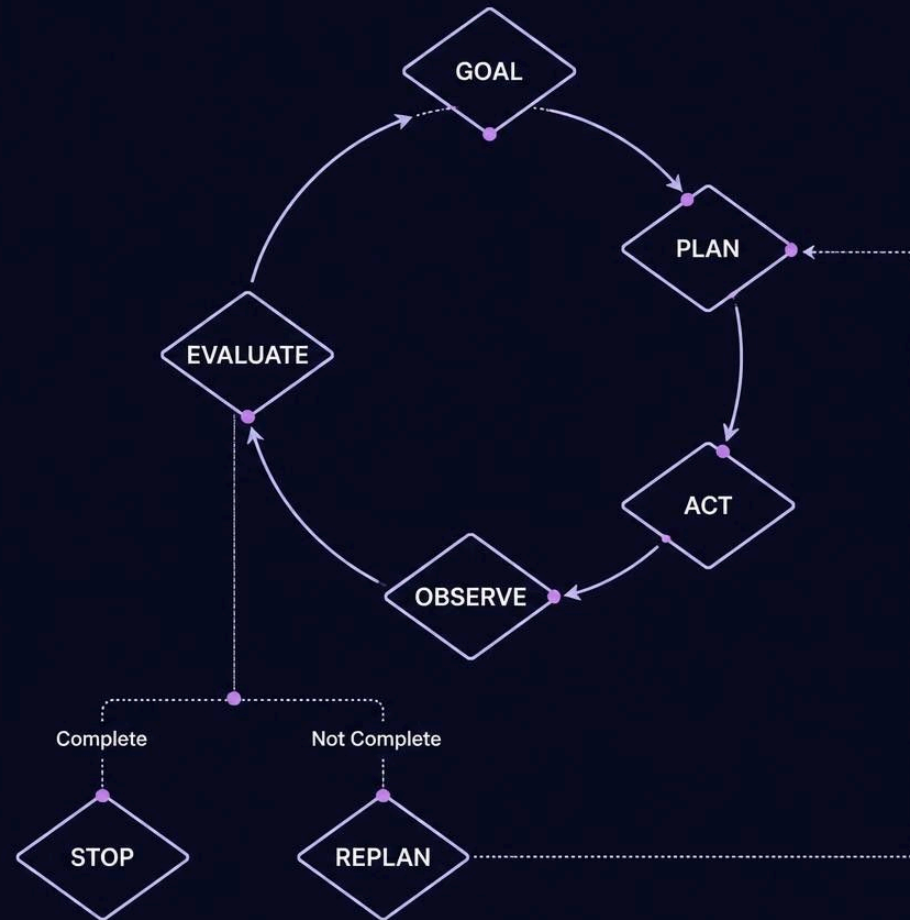
3. Incremental Planning

Plan only the next meaningful set of actions, then re-evaluate. Maximizes flexibility but can lose sight of the overall goal without careful design.

❓ Which strategy is appropriate for an uncertain enterprise workflow where external systems can fail, data can be missing, and approvals can be delayed?

The Agent Planning Loop

This is the central control loop of a reasoning agent. Every sophisticated agent architecture — regardless of framework — implements some variation of this cycle.



The loop is not a failure of planning — it is the intended design. Real-world tasks require iteration. An agent that cannot replan is brittle by design.

Reasoning Is Not Just "Thinking"

The word "reasoning" is often used loosely. In agent systems, it is more useful to decompose it into distinct cognitive operations — each of which can be implemented, tested, and evaluated separately.



Reasoning

Choosing what should happen next given the current state. The LLM's core contribution.



Planning

Creating a sequence of future actions to achieve the goal.



Execution

Actually performing those actions — calling tools, writing to systems.



Observation

Capturing new information from the environment after each action.



Reflection

Evaluating what happened and deciding what to do differently.

Reflection

Reflection is what turns a single-pass executor into an adaptive agent. After each action, the agent evaluates the outcome against the intended goal and determines whether to proceed, correct, or escalate.



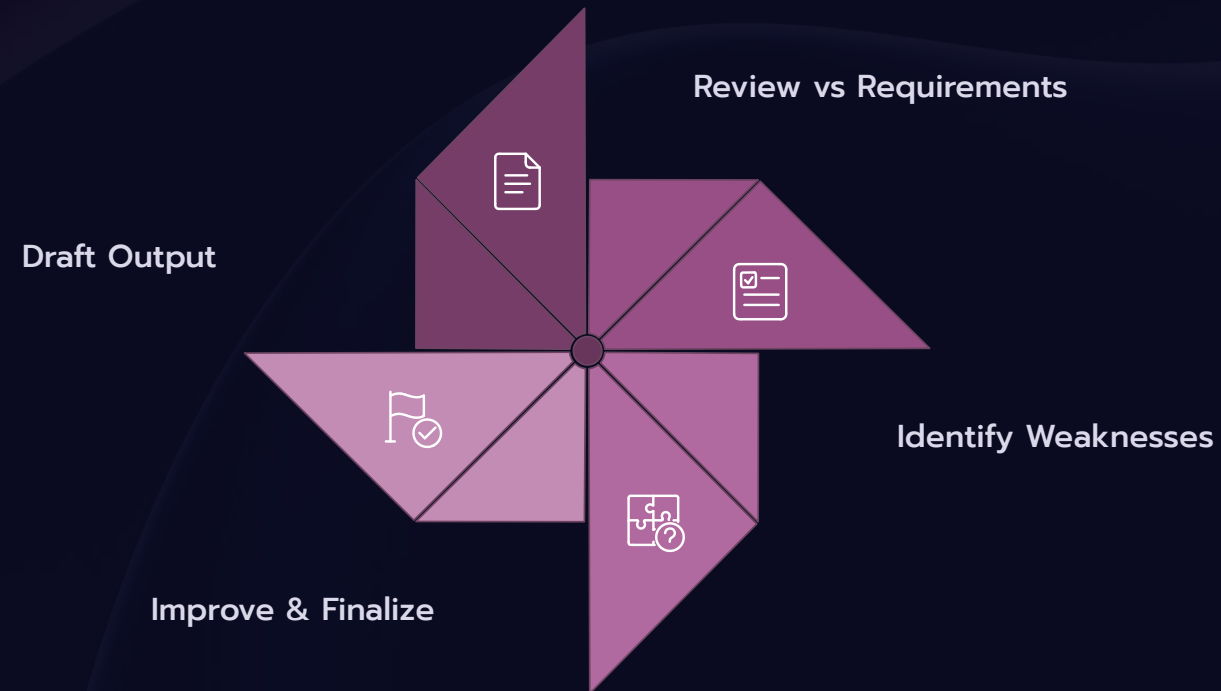
Why reflection matters

Without reflection, the agent would submit a defective checklist and move on. Reflection **creates an opportunity to detect and correct errors before they propagate** downstream through the workflow.

Reflection is most powerful when paired with structured state — the agent can compare what it intended to produce against what it actually produced.

Self-Critique

Self-critique is a structured form of reflection where the agent explicitly reviews its own output against a set of requirements before considering a task complete.



"Review this employee onboarding plan against the stated requirements. Identify any missing steps, compliance gaps, or incorrect assumptions."

⚠ Important distinction: Self-critique improves output quality but is probabilistic. It is *not* a substitute for deterministic validation. Use code-based checks for anything that must be verified with certainty.

Replanning

The plan is a hypothesis about how to achieve the goal. When reality contradicts that hypothesis, the agent must revise — not blindly continue. The ability to replan is what separates a resilient agent from a brittle script.

Original plan

1. Assign laptop
2. Create system accounts
3. Schedule induction session

Observation

 Laptop inventory system is unavailable.

Revised plan

1. Raise hardware request for alternate device
2. Continue account creation (no dependency)
3. Reschedule equipment-dependent tasks
4. Flag laptop dependency for human review



The plan is not sacred. An agent that cannot adapt to failure is not autonomous — it is fragile automation.

Deterministic vs. LLM Reasoning

One of the most important engineering decisions in agent design is knowing **when not to use an LLM**. Every task that can be solved deterministically should be. Reserve the LLM for tasks that genuinely require language understanding or flexible reasoning.

Task	Preferred approach
Add two numbers	Python / code
Validate email format	Python / code
Check authorization rules	Python / code
Calculate SLA deadline	Python / code
Decide next task given ambiguity	LLM
Interpret an ambiguous user request	LLM
Generate a plan from a goal	LLM
Summarize observations	LLM

⊗ Don't use an LLM where ordinary software is faster, cheaper, and more reliable.

Managing Large Contexts

As agent workflows grow in complexity, context size becomes a first-class engineering problem. Left unmanaged, it leads to higher cost, slower responses, and degraded reasoning quality.

The problem compounds quickly

100+

Dialogue turns

50+

Tool results

20+

Documents loaded

Management techniques

- **Summarization**
Compress older turns into rolling summaries.
- **Selective retrieval**
Fetch only what is relevant to the current step.
- **Structured state**
Replace raw history with a compact state object.
- **Tool result truncation**
Return only the fields the agent actually needs.
- **Hierarchical memory**
Tier information by recency and relevance.

Context Failure Modes

Getting context wrong is one of the most common sources of agent failure. These four failure modes cover the vast majority of context-related bugs in production systems.

Too Little Context

The agent lacks critical information and makes decisions based on incomplete evidence. Leads to incorrect or incomplete outputs.

Too Much Context

The model becomes distracted by irrelevant information. Attention dilutes and reasoning quality degrades — even with a large context window.

Wrong Context

The agent reasons over information that is not relevant to the current task. Retrieval errors are a common cause.

Stale Context

The agent acts on outdated information — a system status that has changed, a record that has been updated, or a decision that has been superseded.



These failure modes connect directly to the railway-agent lab — you have likely encountered at least two of them already.

Memory Failure Modes

Memory adds power but also risk. Without deliberate governance, memory systems accumulate problems that are difficult to detect and expensive to fix in production.

Storing everything

Unbounded memory growth degrades retrieval quality and increases storage cost.

Retrieving irrelevant information

Poor retrieval injects noise into context, degrading reasoning quality.

Stale or contradictory memories

Outdated facts can override correct current information. Conflicting memories cause inconsistent behavior.

Privacy and security exposure

Sensitive data written to memory without proper access controls creates compliance and security risk.

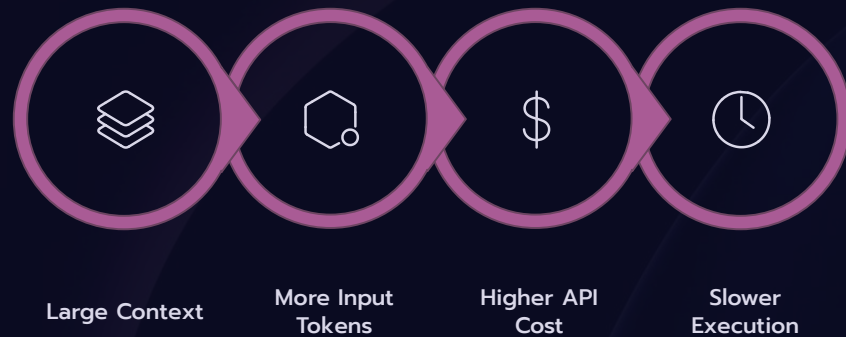
Treating retrieved info as authoritative

Retrieved memory should be validated — it may be stale, incomplete, or incorrect.

⊗ **Memory needs governance.** Define retention policies, access controls, and expiry rules from the start.

Cost Optimization

Context is not free. Every input token has a cost. At production scale, poor context management directly translates into higher API bills, slower responses, and worse user experience.

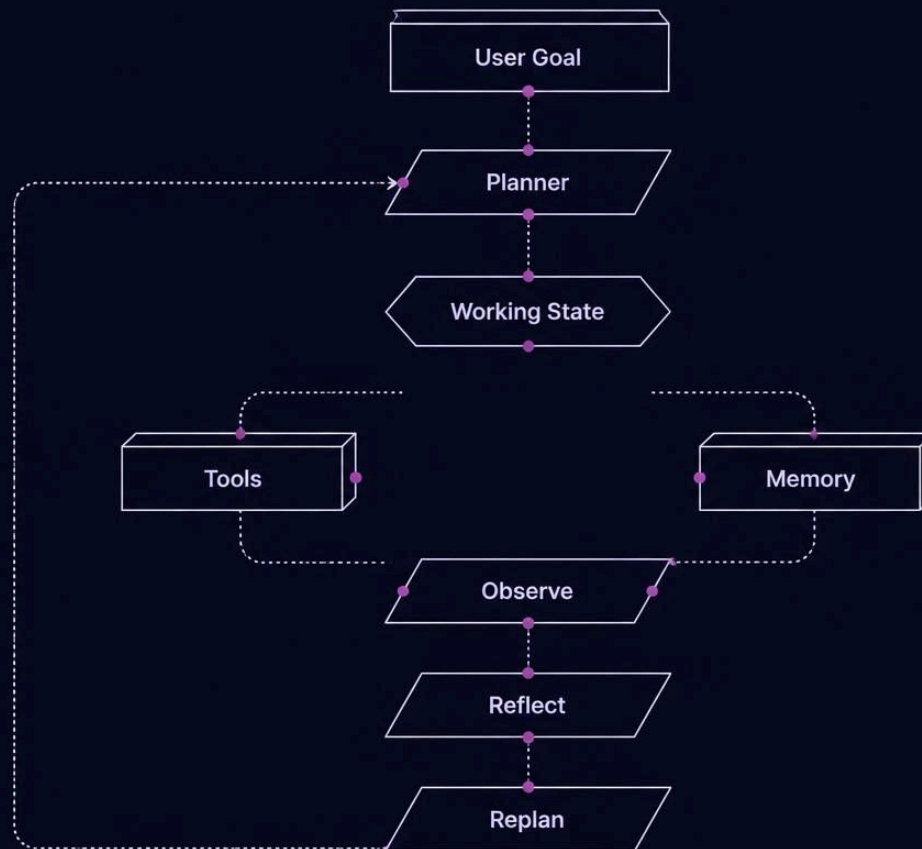


Optimization techniques

- **Concise system prompts**
Avoid padding; every word costs tokens.
- **Summarize old conversations**
Replace verbose history with compact summaries.
- **Retrieve selectively**
Only include memory directly relevant to the step.
- **Minimize tool output**
Return only the fields the agent needs.
- **Use smaller models where appropriate**
Not every step requires the most capable — and expensive — model.

A Practical Architecture

Pulling the concepts together: a complete agent architecture that integrates planning, working state, memory, observation, and reflection into a coherent, auditable loop.



Each layer has a clear responsibility. The working state object is the source of truth. Tools and memory are read/write resources. Observation, reflection, and replanning are the control mechanisms.

Mini Demo: Memory-Enabled Onboarding Agent

Watch how a structured working memory transforms the agent's ability to answer questions without reconstructing everything from scratch.

Initial request

"Onboard Ramesh as a Senior Python Engineer."

Agent gathers structured state

Employee details

Name, role, location, manager

Equipment needs

Laptop model, peripherals, badge

Access & training

Systems, repositories, mandatory modules

Then ask:

"What is still pending?"

The agent answers **instantly from its structured working memory** — it does not re-read conversation history, re-call tools, or re-reason from scratch. It simply reads the `pending_steps` field from the state object.

This is a dramatically more reliable pattern than asking the LLM to remember details across a long conversation.



Demo: Replanning in Action

This demonstrates why replanning is a necessary capability — not an edge case. Enterprise workflows routinely encounter failures that a static plan cannot handle.

Initial plan

1. Validate employee record
2. Assign laptop from inventory
3. Create system access
4. Schedule induction

Injected failure

⚠ Laptop inventory system is unavailable.

Observe → Update State → Revise Plan

01

Detect failure

Laptop step returns error; state updated to `laptop: blocked`

02

Update state

Mark dependent tasks as deferred

03

Revise plan

Raise hardware request; continue independent tasks

04

Continue

Create accounts and schedule induction in parallel

📌 This is a much stronger demonstration than a chatbot remembering a name. It shows true adaptive autonomy.

Lab 3: Enterprise Workflow Agent

In Lab 3, participants build a complete autonomous agent by combining every concept introduced in this deck. The target workflow is **Employee Onboarding** — a realistic enterprise process with dependencies, approvals, and failure modes.



Memory & State

Structured working state object with persistence



Tools

Multiple integrated tool calls with error handling



Validation & Guardrails

Deterministic checks on critical steps



Human Approval

Approval gates for sensitive actions



Logging

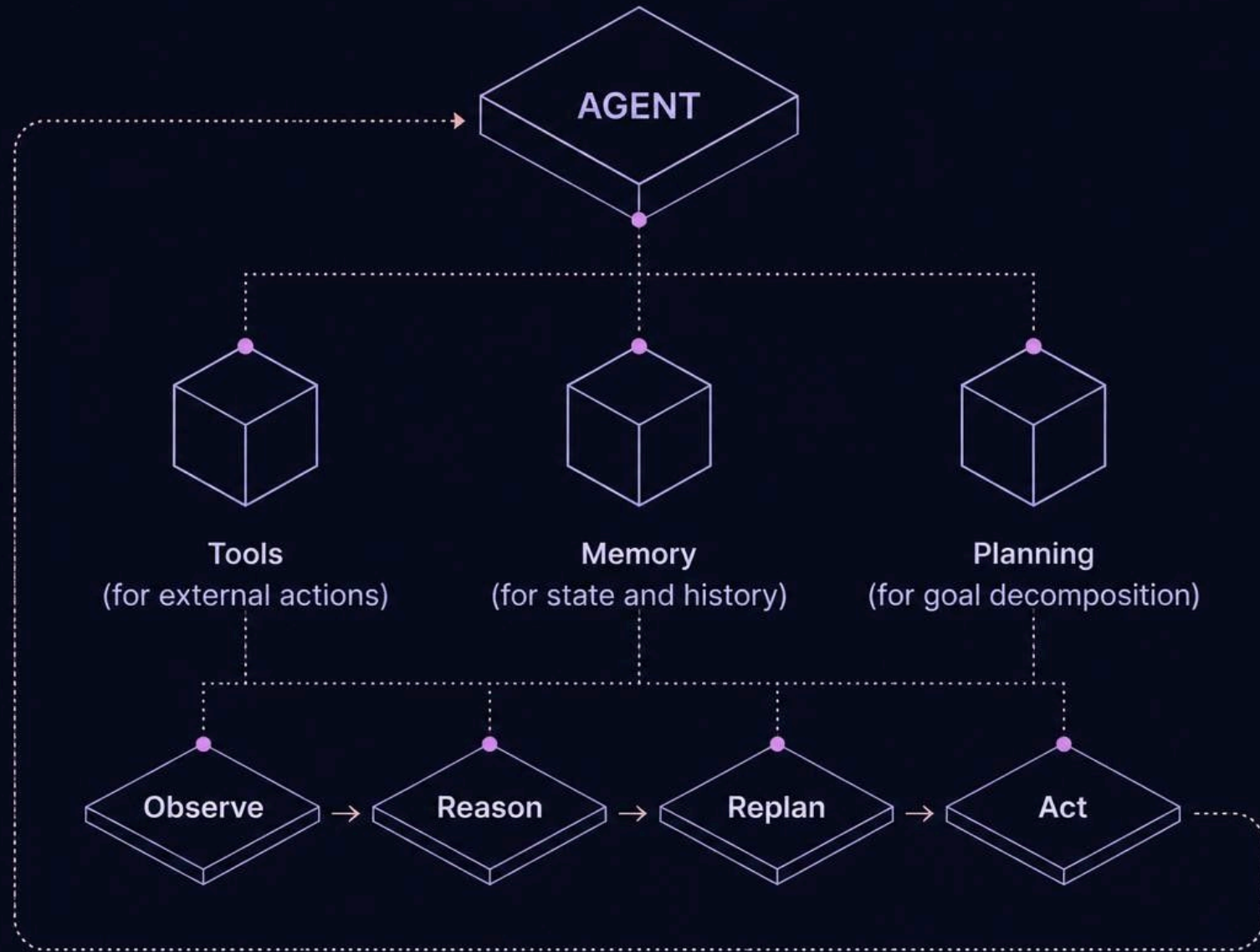
Full audit trail of decisions and actions



Outcome: A production-grade autonomous enterprise workflow agent — not a chatbot.

The Big Picture

Step back and look at what we have now assembled. A modern agent is not simply an LLM with a prompt. It is a system of interacting capabilities, each playing a distinct role in achieving complex goals reliably.





The Design Question

"What should the agent remember, what should it retrieve, and what should it decide?"

This is the central design question for every agent workflow. There is no universal answer — it depends on your business requirements, data sensitivity, latency constraints, and cost budget.

1

Map your workflow

Identify every piece of information the agent touches. What is ephemeral? What must persist?

2

Classify each memory type

Assign each information type to the appropriate store: context, working state, or long-term memory.

3

Define decision boundaries

Which decisions must be deterministic? Which require LLM reasoning? Which require human approval?

Discuss with your neighbor: apply this question to your own enterprise workflow.

Next → Enterprise Agent Architecture & Multi-Agent Patterns

We have built an agent that can reason, use tools, maintain state, remember relevant information, plan, observe, reflect, and replan. That is a significant capability. But some problems are too large, too parallel, or too specialized for a single agent.

What we've built in Deck 4

- Reasoning
- Tool calling
- State management
- Long-term memory
- Planning & replanning
- Observation & reflection

The next question

What happens when one agent is no longer enough?

When tasks are too complex, too parallel, or require specialized expertise — you need agents that coordinate with other agents.

Deck 5: Enterprise Agent Architecture & Multi-Agent Patterns